# Dynamic Memory Allocation

**Introduction**.....1
**Memory Layout of C Program**.....2
**1. malloc()**.....3
**2. calloc()**.....5
**3. realloc()**.....7
**4. free()**.....9
Memory Leak.....10
**Fragmentation**.....14
**Types of Fragmentation**.....14
**Advantages of Dynamic Memory Allocation**.....16
Disadvantages.....16
**Practice Questions**.....17
**Programming Questions**.....17

# Introduction

Memory management is one of the most important concepts in C programming.

**In normal variable declaration: int a;**

Memory is allocated automatically by the compiler and this is called static Memory Allocation.

But sometimes we do not know memory requirement in advance.

**Examples:**

- Number of students entered by user
- Dynamic arrays
- Linked lists
- Queues
- Trees

In such cases, memory is allocated during program execution.This is called **Dynamic Memory. Allocation (DMA).**

### Definition

Dynamic Memory Allocation is the process of allocating memory during runtime using library functions.

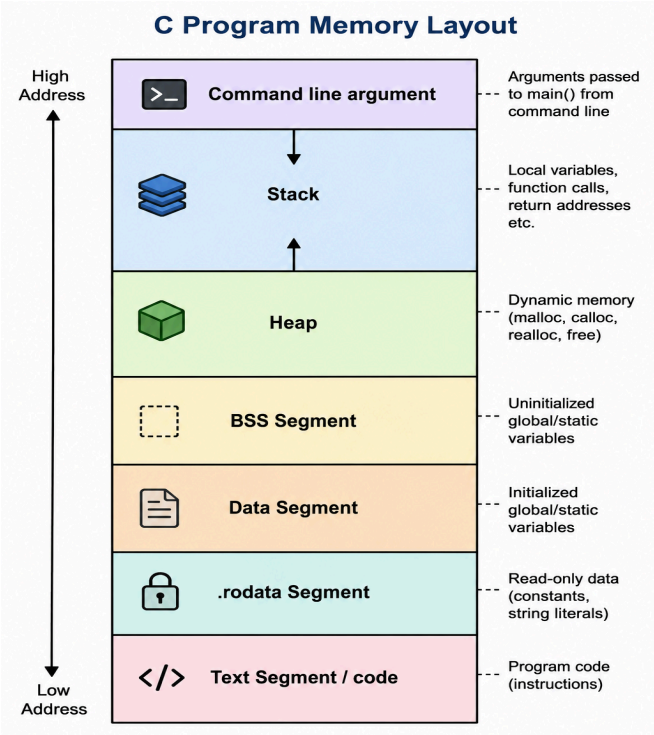
These functions are available in: **#include <stdlib.h>**

### Dynamic Memory Allocation Functions

| Function | Purpose |
|---|---|
| malloc() | Allocates single block of memory |
| calloc() | Allocates multiple blocks and initializes to zero |
| realloc() | Changes size of allocated memory |
| free() | Releases allocated memory |

# Memory Layout of C Program

A C program memory is divided into different sections.



### Heap Memory

Dynamic memory allocation occurs in a **Heap Area.** Heap memory is managed manually by programmer.

### Stack vs Heap

| Stack | Heap |
|---|---|
| Automatic allocation | Manual allocation |
| Faster | Slower |
| Fixed size | Dynamic size |
| Managed by compiler | Managed by programmer |
| Function variables | Dynamic memory |

# 1. malloc()

Meaning is **memory allocation**, Used to allocate memory dynamically.

**Syntax:**
ptr = (datatype*) malloc(size_in_bytes);

### Example

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;

    // Allocate memory for 5 integers dynamically
    ptr = (int*) malloc(5 * sizeof(int));
```

```c
    // Check whether memory allocation was successful
    if(ptr == NULL)
    {
        printf("Memory not allocated");
        return 0;
    }

    printf("Memory allocated");

    // Release the dynamically allocated memory
    free(ptr);

    return 0;
}
```

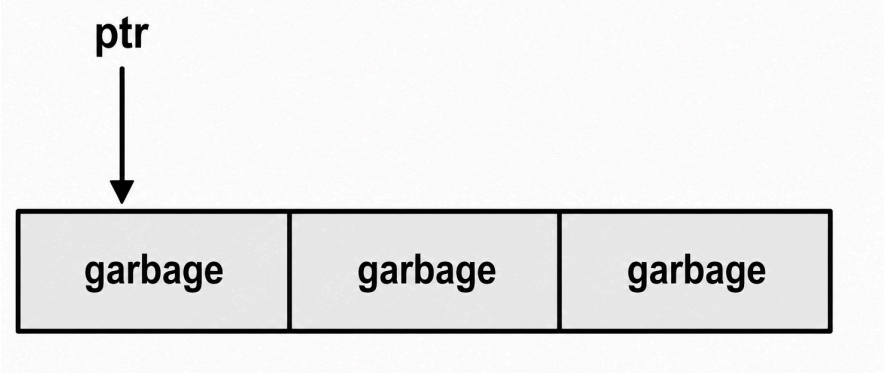
## Explanation

malloc(5 * sizeof(int)) it  allocates memory for 5 integers. If integer size is 4 bytes:  5 × 4 = 20 bytes and memory is allocated on  heap.

### Important Properties of malloc()

1. Allocates contiguous memory
2. Returns void*
3. Memory contains garbage values
4. Returns NULL if allocation fails

## Memory Representation




## Why Typecasting?

malloc() returns a void* pointer which is generic pointer. Typecasting converts it into required pointer type.

# 2. calloc()

`calloc()` stands for **Contiguous Allocation**. It is a **dynamic memory allocation function** provided by `<stdlib.h>`.

It allocates memory for **multiple elements** and initializes all allocated bytes to **zero**.

**Syntax:**

ptr = (datatype*) calloc(number_of_blocks, size_of_each_block);

### Example

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;

    // Allocate memory for 5 integers and initialize them to zero
    ptr = (int*) calloc(5, sizeof(int));

    // Check whether memory allocation was successful
    if(ptr == NULL)
    {
        printf("Memory allocation failed");
        return 0;
    }

    // Print the allocated elements
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", ptr[i]);
    }

    // Free the dynamically allocated memory
    free(ptr);

    // Set pointer to NULL after freeing the memory
```

```
    ptr = NULL;

    return 0;
}
```

## Output

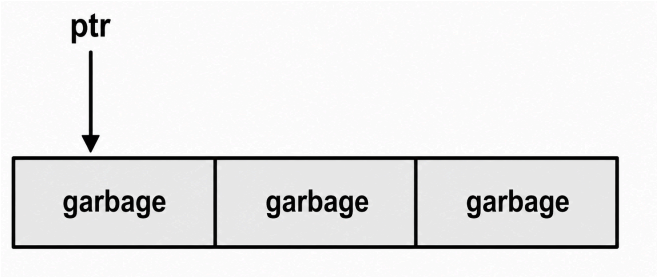
0 0 0 0 0

## Important Difference Between malloc() and calloc()

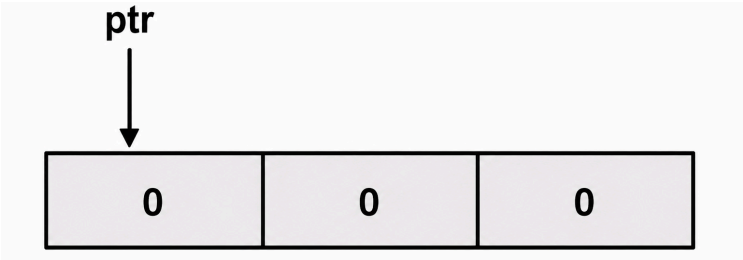
| malloc() | calloc() |
|---|---|
| Single argument | Two arguments |
| Garbage values | Initializes to zero |
| Faster | Slightly slower |

## Internal Memory

### 1) malloc()




### 2) calloc()



# 3. realloc()

`realloc()` means **Re-allocation**. It is used to **change the size of memory that was previously allocated dynamically** using `malloc()`, `calloc()`, or `realloc()`.

### Syntax

ptr = realloc(ptr, new_size);
Example

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *ptr;

    // Allocate memory for 3 integers
    ptr = (int*) malloc(3 * sizeof(int));

    // Store values in the allocated memory
    ptr[0] = 10;
    ptr[1] = 20;
    ptr[2] = 30;

    // Resize the memory to hold 5 integers
    ptr = (int*) realloc(ptr, 5 * sizeof(int));

    // Store additional values
    ptr[3] = 40;
    ptr[4] = 50;

    // Print all 5 elements
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", ptr[i]);
    }

    // Free the dynamically allocated memory
    free(ptr);
```

```c
    // Set pointer to NULL after freeing the memory
    ptr = NULL;

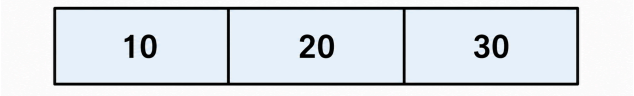
    return 0;
}
```

## Output

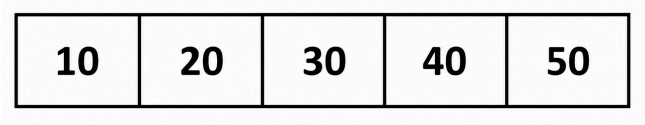
10 20 30 40 50

**What realloc() Does Internally?**

Suppose current memory:




Need larger memory:




If adjacent memory is free:

- Expands same block

Otherwise:

- Allocates new block
- Copies old data
- Frees old memory

## Important Notes

| Case | Behavior |
|---|---|
| realloc(ptr, bigger_size) | Expands memory |
| realloc(ptr, smaller_size) | Shrinks memory |
| realloc(ptr, 0) | Equivalent to free |

| | |
|---|---|
| realloc(NULL, size) | Equivalent to malloc |

# 4. free()

`free()` is a dynamic memory management function in C used to release memory that was previously allocated using `malloc()`, `calloc()`, or `realloc()`. It returns the allocated memory back to the heap so that it can be reused by the program. The syntax is `free(ptr);`, where `ptr` points to the dynamically allocated memory. After freeing the memory, it is good practice to assign `NULL` to the pointer using `ptr = NULL;` to prevent it from becoming a dangling pointer. We should never access or free the same memory again after it has been released. `free(NULL)` is safe and does nothing.

### Syntax
free(ptr);

### Example
```
int *ptr;

ptr = (int*) malloc(10 * sizeof(int));

free(ptr);
```

#### Why free() Important?

If memory is not released **Memory Leak occurs.**

# Memory Leak

A memory leak occurs when a program dynamically allocates memory but loses the pointer to that memory without freeing it. As a result, the allocated memory remains occupied but the program can no longer use or release it. If this happens repeatedly, the available memory gradually decreases and may eventually cause the program to run out of memory.

### Definition:
A memory leak occurs when a program keeps holding onto memory that it no longer uses, making that memory unavailable for other purposes.

## Example of Memory Leak

```
while(1)
{
   malloc(1000);
}
```

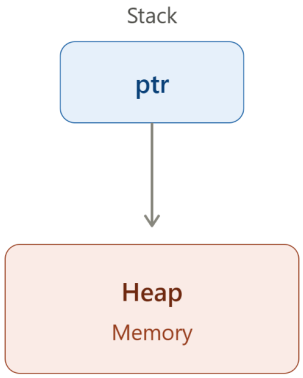
Heap memory keeps increasing and the program may crash.

## What Happens After free()?

Suppose a pointer `ptr` points to dynamically allocated memory in the **heap**:

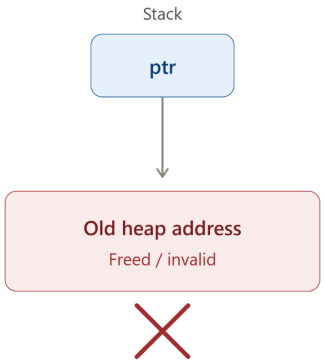
int *ptr = malloc(sizeof(int));

At this point:




When we release the memory using free(ptr); the heap memory is **returned to the memory manager and becomes available for reuse**.

However, the pointer variable `ptr` itself is **not automatically changed to `NULL`**. It still contains the old memory address.




Memory is no longer owned by ptr. If we try to access the memory using `ptr` after `free(ptr)`, the behavior is **undefined** because that memory is no longer valid for use by `ptr`. Such a pointer is called a **dangling pointer**.

**Important point**

To avoid a dangling pointer, set the pointer to `NULL` after freeing the memory.

```
free(ptr);
ptr = NULL;
```

## Dangling Pointer Example

```
int *ptr;

ptr = (int*) malloc(sizeof(int));

free(ptr);

printf("%d", *ptr);
```

Undefined behavior.

## Safe Practice

After free()  assigne ptr = NULL;

## Double free() Problem

## What Happens if free() is Called Two Times?

**Example:**

```
int *ptr;
ptr = (int*) malloc(sizeof(int));

free(ptr);

free(ptr);
```

This Causes a **undefined behavior.**

Possible outcomes:

- Program crash
- Heap corruption
- Runtime error
- Security vulnerability

## Why Double free() is Dangerous?

When memory is freed:

- Heap manager marks block as available

Second free():

- Tries to free already freed block
- Corrupts heap management structure

# Internal Working of Dynamic Memory Allocation

When malloc() is called: malloc(100);

memory allocator performs:

1. Searches free heap block
2. Reserves requested memory
3. Stores metadata
4. Returns usable address

## What is Metadata?

Metadata means, Information about allocated memory block stored internally by memory allocator.

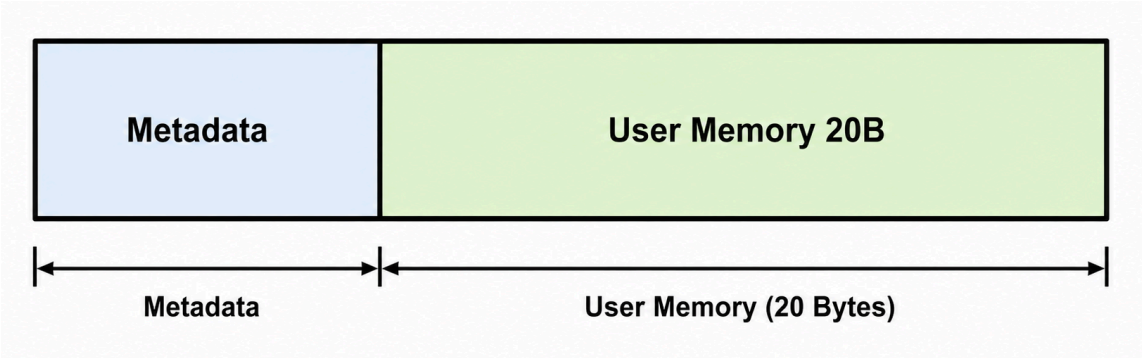
### Metadata Contains

| Information | Purpose |
|---|---|
| Block size | Size of allocated memory |
| Allocation status | Free or allocated |
| Next block pointer | Heap management |
| Previous block pointer | Memory linking |

## Actual Heap Structure

When user requests a malloc(20);

Actual memory allocated internally:




### Example Metadata

Size = 20 bytes
Allocated = Yes
Next Block Address

**Note : Programmer can access only user memory metadata is hidden.**

### How free() Works Internally?

When user use free(ptr);

allocator:

1. Reads metadata
2. Marks block free
3. Merges adjacent free blocks
4. Updates heap structure

# Fragmentation

**Fragmentation** is a condition where the available memory becomes broken into many **small, scattered free blocks** due to repeated **memory allocation (malloc())** and **deallocation (free())**.

Although the **total free memory may be enough**, it is **not available as one continuous block**, so a large allocation request may fail.

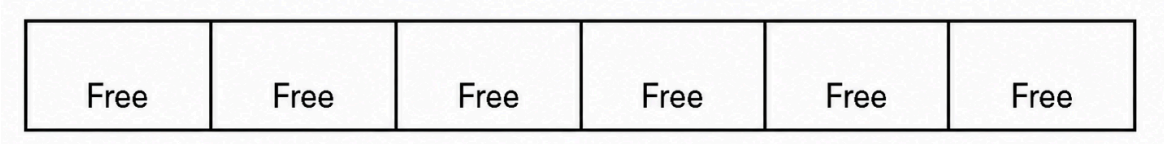
## Types of Fragmentation

| Type | Meaning |
|---|---|
| Internal Fragmentation | Extra unused memory inside block |
| External Fragmentation | Free memory scattered |

### How Fragmentation Occurs

Suppose the heap initially looks like this:

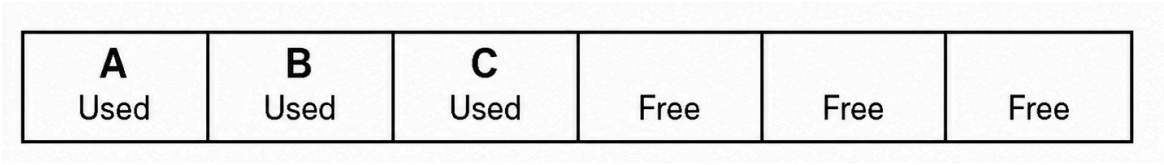
Heap Memory




#### Step 1: Allocate three blocks

A = malloc(100);
B = malloc(100);

C = malloc(100);




### Step 2: Free the middle block

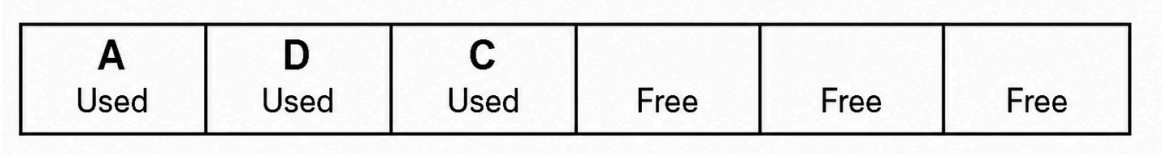
free(B);




Now the free memory is split into separate pieces.

### Step 3: Allocate another smaller block

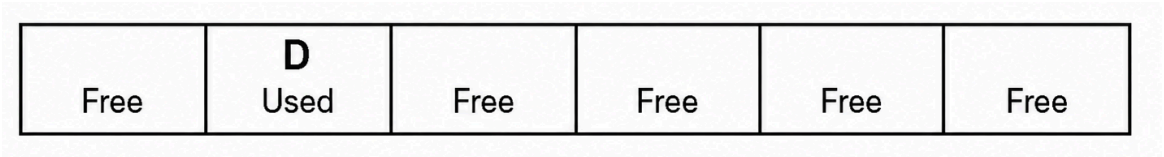
D = malloc(50);




The remaining unused space in that block may become another tiny free block.

### Step 4: Free A and C

free(A);
free(C);




Now memory is broken into several separate free blocks.

Suppose a program requests malloc(250);

The allocation may **fail** because there is **no single contiguous free block of 250 bytes**, even though the **total free memory is greater than 250 bytes**.

This is called **fragmentation**.

# Advantages of Dynamic Memory Allocation

1. Efficient memory usage
2. Memory allocated as needed
3. Supports dynamic data structures
4. Reduces wastage

# Disadvantages

1. Slower than static allocation
2. Memory leaks possible
3. Dangling pointers
4. Fragmentation
5. Manual management required

## Best Practices while allocating memory dynamically

1. Always check NULL
2. Always free allocated memory
3. Set pointer to NULL after free
4. Avoid double free
5. Avoid accessing freed memory

## Table

| Function | Purpose |
|---|---|
| malloc() | Allocate memory |
| calloc() | Allocate and initialize to zero |
| realloc() | Resize memory |
| free() | Release memory |

# Practice Questions

1. Difference between malloc() and calloc()
2. What is dangling pointer?

3. What is memory leak?
4. What happens after free()?
5. What happens if free() is called twice?
6. What is metadata in heap memory?
7. Difference between stack and heap
8. How realloc() works internally?
9. What is fragmentation?
10. Why should pointer be set to NULL after free()?

# Programming Questions

1. Dynamic Integer

Write a C program to:

- Allocate memory for one integer using malloc().
- Read a number from the user.
- Display the entered number.
- Free the allocated memory.

**Concepts:** malloc(), pointer, free()

2. Dynamic Float Array

Write a C program to:

- Dynamically allocate memory for **N floating-point numbers**.
- Read the elements.
- Find and display the average.

**Concepts:** Dynamic array, pointer arithmetic

3. Sum of Dynamic Array

Write a program to:

- Read the size of an integer array at runtime.
- Allocate memory dynamically.
- Find the sum and average.

**Concepts:** Arrays, pointers, DMA

4. Resize an Array

Write a program to:

- Allocate memory for 5 integers.
- Read values.
- Increase the size to 10 integers using realloc().
- Read remaining values.
- Display all elements.

**Concepts:** realloc()

5. Dynamic Character Array

Write a program to:

- Dynamically allocate memory for a string.
- Read the string.
- Display its length without using strlen().

**Concepts:** Character pointers, strings

6. Reverse Dynamic Array

Write a program to:

- Dynamically allocate an array.
- Reverse its elements using pointer arithmetic.
- Display the reversed array.

7. Maximum Element

Write a program that:

- Dynamically allocates an integer array.
- Finds the maximum element using only pointers.

8. Matrix using Dynamic Memory

Write a program to:

- Dynamically allocate a **2D matrix**.
- Read matrix elements.
- Display the matrix.

**Concepts:** Pointer to pointer (int **)

9. Matrix Addition

Write a program to:

- Dynamically allocate two matrices.
- Perform matrix addition.
- Display the resultant matrix.

10. Dynamic Array of Pointers

Write a program to:

- Dynamically allocate memory for an array of integer pointers.
- Store values.
- Print the values.

11. Dynamic Structure

Create a structure:

```
struct Student
{
  int roll;
  char name[20];
  float marks;
};
```

Write a program to:

- Dynamically allocate memory for one student.
- Read the details.
- Display them.

12. Array of Structures

Write a program to:

- Dynamically allocate memory for **N students**.
- Read student details.
- Display all records.

13. Highest Marks

Using dynamic allocation for an array of structures:

- Read details of N students.
- Display the student having the highest marks.

14. Employee Database

Create an Employee structure.

The program should:

- Allocate memory dynamically.
- Read employee details.
- Search an employee using Employee ID.

15. Dynamic Nested Structure

Create structures:

```
struct Address
{
  char city[20];
  int pin;
};

struct Student
{
  int roll;
  char name[20];
  struct Address addr;
};
```

Allocate memory dynamically and display the entered details.

16. Pointer Arithmetic

Write a program to:

- Dynamically allocate memory for N integers.
- Print array elements using:
    - array indexing
    - pointer notation

Compare both outputs.

17. Array of Strings

Write a program to:

- Dynamically allocate memory for **N strings**.
- Read and display all strings.

18. Double Pointer

Write a program that:

- Dynamically allocates memory using a double pointer.
- Stores integer values.
- Displays the values.

19. Library Management

Create a structure:

```
struct Book
{
  int id;
  char title[30];
  float price;
};
```

Write a program to:

- Dynamically allocate memory for N books.
- Display the most expensive book.

20. Student Result Management

Create a program that:

- Dynamically allocates memory for N students.
- Stores roll number, name, and marks.
- Calculates:
    - Total students
    - Average marks
    - Highest marks
    - Lowest marks
- Displays all details.

21. Dynamic Jagged Array

Write a program to create a jagged array where each row contains a different number of elements.

22. Merge Two Dynamic Arrays

Allocate two arrays dynamically.
Merge them into a third dynamically allocated array.

23. Dynamic Polynomial

Store polynomial coefficients dynamically.
Evaluate the polynomial for a given value of **x**.

24. Sparse Matrix using Dynamic Memory

Represent a sparse matrix using dynamically allocated structures and display its triplet representation.

25. Menu-Driven Student Database

Develop a menu-driven program using dynamic memory allocation that supports:

1. Add student
2. Delete student
3. Update student
4. Search student
5. Display all students
6. Exit

Use malloc(), realloc(), and free() appropriately.